Java TreeMap

The Definitive Guide to Sorted Maps

CODEHIVE

1. What is a TreeMap?

A **TreeMap** is a map implementation that stores **Key-Value pairs** in a specific sorted order. Unlike a HashMap, which uses hashing to place items randomly, the TreeMap uses a **Red-Black Tree** structure to keep its keys organized.

The TreeMap Philosophy:

- **Natural Sorting:** Keys are automatically sorted in ascending order (A-Z, 1-10).
- **No Null Keys:** Because it compares keys to sort them, a `null` key would cause a crash.
- **Unique Keys:** Just like all Maps, keys cannot be duplicated.

2. Creating your TreeMap

To use a TreeMap, import it from the `java.util` package. It is standard practice to declare it using the `Map` interface to maintain flexibility.

```
import java.util.Map;
import java.util.TreeMap;

Map<String, String> capitals = new TreeMap<>();
```

By using `Map` on the left side, you can easily switch to a `HashMap` later if you decide that sorting is no longer necessary, without changing the rest of your code.

3. Populating the Map

We use the `put()` method to add data. Notice that even if we add countries out of order, the TreeMap will reorganize them internally.

```
capitals.put("India", "New Delhi");
capitals.put("Austria", "Wien");
capitals.put("Norway", "Oslo");
capitals.put("USA", "Washington DC");
capitals.put("England", "London");
```

Resulting Order: Austria → England → India → Norway → USA.

4. Reading Values

To get a value, simply provide the Key. If the key is not present, Java returns `null`.

```
// Accessing a value
String city = capitals.get("Austria");
System.out.println(city); // Wien
```

Checking Map Size

The `size()` method tells you exactly how many sorted pairs exist in the tree.

```
System.out.println(capitals.size()); // 5
```

5. Modifying the Collection

Removing items is straightforward. You can remove a specific key or wipe the entire structure.

```
// Removing one entry
capitals.remove("USA");

// Clearing everything
capitals.clear();
```

Pro Tip: Removing an item from a TreeMap takes O(log n) time because the tree has to re-balance itself to remain efficient.

6. Looping Through Keys

When you loop through a TreeMap, you are guaranteed to receive the data in sorted order. This makes it perfect for generating alphabetical lists.

```
// Print sorted keys
for (String country : capitals.keySet()) {
    System.out.println(country);
}
```

7. Printing Pairs

The most common way to display a TreeMap is by iterating through the `keySet` and fetching the `get()` value, or using `values()` directly.

```
// Print keys and values together
for (String key : capitals.keySet()) {
    System.out.println("Country: " + key +
                       " | Capital: " + capitals.get(key));
}
```

8. The 'var' Keyword

Introduced in Java 10, the `var` keyword helps reduce redundancy in your code while maintaining strong typing.

```
// Old way
TreeMap<String, String> t = new TreeMap<>();

// Modern way
var sortedCapitals = new TreeMap<String, String>();
```

9. TreeMap vs HashMap

The choice between these two determines the performance profile of your application.

Feature	HashMap	TreeMap
Order	Unpredictable	Alphabetical / Natural
Null Keys	Allowed (1)	Forbidden
Speed	O(1) - Fast	O(log n) - Moderate